MDTS-4214 Assignment-5

Subhayan Biswas

February 26, 2026

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1 Problem Set 3:

1.1 3. Problem to demonstrate the role of qualitative (ordinal) predictors in addition to quantitative predictors in multiple linear regression

Consider `diamonds` data set in R. It is in the ggplot2 package. Make a list of all the ordinal categorical variables. Identify the response.
(a) Run a linear regression of the response on the quality of cut. Write the fitted regression model.
(b) Test whether the expected price of diamond with premium cut is significantly different from that of the ideal cut.
(c) What is the expected price of a diamond of ideal cut?
(d) Modify the regression model in (a) by incorporating the predictor "table". Write the fitted regression model.
(e) Test for the significance of "table" in predicting the price of diamond.
(f) Find the average estimated price of a diamond with an average table value and which is of fair cut.

[3]:
```
import statsmodels.api as sm
df = sm.datasets.get_rdataset('diamonds', 'ggplot2').data
df.head()
```

[3]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Among all the features or covariates mentioned in the `diamonds` dataset, the two variables `cut` and `clarity` appear to be Ordinal variables.
The variable which should be dependent on the other variables of this dataset is `price`. Thus `price` is the response here.

[4]:
```
from statsmodels.formula.api import ols
model1 = ols('price ~ cut', data=df).fit()
print(model1.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  price   R-squared:                       0.013
Model:                            OLS   Adj. R-squared:                  0.013
Method:                 Least Squares   F-statistic:                     175.7
Date:                Thu, 26 Feb 2026   Prob (F-statistic):          8.43e-150
Time:                        22:37:20   Log-Likelihood:            -5.2343e+05
No. Observations:               53940   AIC:                         1.047e+06
Df Residuals:                   53935   BIC:                         1.047e+06
Df Model:                           4
Covariance Type:            nonrobust
================================================================================
====
                     coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
----
Intercept         4358.7578     98.788     44.122      0.000    4165.133
4552.383
cut[T.Good]       -429.8933    113.849     -3.776      0.000    -653.039
-206.748
cut[T.Ideal]      -901.2158    102.412     -8.800      0.000   -1101.943
-700.488
cut[T.Premium]     225.4999    104.395      2.160      0.031      20.884
430.115
cut[T.Very Good]  -376.9979    105.164     -3.585      0.000    -583.121
-170.875
==============================================================================
Omnibus:                    15109.877   Durbin-Watson:                   0.033
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            34437.824
Skew:                           1.615   Prob(JB):                         0.00
Kurtosis:                       5.212   Cond. No.                         14.9
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

In the light of our derivations, the fitted regression model of `price` on quality of cut is given by: $\hat{price} = 4358.7578 - 429.8933Good - 376.9979VeryGood + 225.4999Premium - 901.2158Ideal$

[10]:
```
from statsmodels.iolib.summary2 import summary_col
print(summary_col([model1], stars=True, float_format='%0.2f'))
```

```
============================
                     price
----------------------------
Intercept          4358.76***
                   (98.79)
cut[T.Good]        -429.89***
                   (113.85)
cut[T.Ideal]       -901.22***
                   (102.41)
cut[T.Premium]     225.50**
                   (104.40)
cut[T.Very Good]   -377.00***
                   (105.16)
R-squared          0.01
R-squared Adj.     0.01
============================
Standard errors in
parentheses.
* p<.1, ** p<.05, ***p<.01
```

b. The 3 stars against the p value corresponding to `Premium` indicates that expected price of diamond with Premium cut is significantly different than that of an Ideal cut.

c. Expected price of a diamond with Ideal cut is the value of intercept term plus the coefficient of `Ideal` of the fitted model,i.e 3457.54 USD.

[15]:
```
model2 = ols('price ~ cut + table', data = df).fit()
print(summary_col([model2], stars=True, float_format='%0.2f'))
```

```
============================
                     price
----------------------------
Intercept          -6218.06***
                   (554.21)
cut[T.Good]        -365.57***
                   (113.50)
cut[T.Ideal]       -345.61***
                   (106.00)
cut[T.Premium]     280.61***
                   (104.07)
cut[T.Very Good]   -180.41*
                   (105.29)
table              179.10***
                   (9.24)
R-squared          0.02
R-squared Adj.     0.02
============================
Standard errors in
```

parentheses.
* p<.1, ** p<.05, ***p<.01

Fitted model: $\hat{price} = -6218.06 - 365.57Good - 180.41VeryGood + 280.61Premium - 345.61Ideal + 179.10table$

e. From the p value corresponding to the predictor `table`, we can say that it is statistically significant in predicting the response `price`.

[18]:
```
t = df['table'].mean()
est_price= -6218.06 + 179.10*t
est_price
```

[18]:
```
np.float64(4072.521637931034)
```

f. Thus the estimated average price of a diamond with an average `table` value and having a fair cut is approximately 4072.52 US dollars.

[19]:
```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error
```

2 Problem Set 5

2.1 Problem to demonstrate the utility of K nearest neighbour regression over least squares regression

Consider a setting with n = 1000 observations. Generate
(i) $x_1 i$ from $N(0, 22)$ and x2i from Poisson($= 1.5$).
(ii) ϵ_i from $N(0, 1)$. (iii) $y_i = -2 + 1.4x_1 i - 2.6x_2 i + \epsilon_i$. Split the data into train and test sets. Keep the first 800 observations as training data and the remaining as test data. Work out the following:
1. Fit a multiple linear regression equation of y on x_1 and x_2. Calculate test MSE.
2. Fit a KNN model with k = 1,2,5,9,15. Calculate test MSE for each choice of k. Suppose the data in Step (iii) is generated as : $y_i = 1/(-2 + 1.4x_1 i - 2.6x_2 i + 2.9x_1 i^2) + 3.1sin(x_2 i) - 1.5x_1 i x_2 i^2 + \epsilon_i$. Work out the problems in (1) and (2). Compare and comment on the results.

[32]:
```
np.random.seed(42)
n = 1000
x1 = np.random.normal(0, 2, n)
x2 = np.random.poisson(1.5, n)
eps = np.random.normal(0, 1, n)
X = np.column_stack((x1, x2))
```

[33]:
```
y_linear = -2 + 1.4*x1 - 2.6*x2 + eps
denom = -2 + 1.4*x1 - 2.6*x2 + 2.9*(x1**2)
y_nonlinear = (1 / denom) + 3.1*np.sin(x2) - 1.5*x1*(x2**2) + eps
```

[34]:
```
X_train, X_test = X[:800], X[800:]
y_lin_train, y_lin_test = y_linear[:800], y_linear[800:]
```

```
y_nonlin_train, y_nonlin_test = y_nonlinear[:800], y_nonlinear[800:]
```

[37]:
```
def evaluate_models_sm(X_train_data, X_test_data, y_train_data, y_test_data,␣
 ↪scenario_name):
    X_train_sm = sm.add_constant(X_train_data)
    X_test_sm = sm.add_constant(X_test_data)
    ols_model = sm.OLS(y_train_data, X_train_sm).fit()
    ols_preds = ols_model.predict(X_test_sm)
    ols_mse = mean_squared_error(y_test_data, ols_preds)
    print(f"OLS Test MSE: {ols_mse:.4f}")

    k_values = [1, 2, 5, 9, 15]
    for k in k_values:
        knn = KNeighborsRegressor(n_neighbors=k)
        knn.fit(X_train_data, y_train_data)
        knn_preds = knn.predict(X_test_data)
        knn_mse = mean_squared_error(y_test_data, knn_preds)
        print(f"KNN (k={k}) Test MSE: {knn_mse:.4f}")
```

[38]:
```
evaluate_models_sm(X_train, X_test, y_lin_train, y_lin_test, "Scenario 1:␣
 ↪Linear DGP")
evaluate_models_sm(X_train, X_test, y_nonlin_train, y_nonlin_test, "Scenario 2:␣
 ↪Non-Linear DGP")
```

```
OLS Test MSE: 0.9946
KNN (k=1) Test MSE: 1.9711
KNN (k=2) Test MSE: 1.6176
KNN (k=5) Test MSE: 1.2978
KNN (k=9) Test MSE: 1.2731
KNN (k=15) Test MSE: 1.3488
OLS Test MSE: 197.4927
KNN (k=1) Test MSE: 23.2432
KNN (k=2) Test MSE: 16.9873
KNN (k=5) Test MSE: 13.9256
KNN (k=9) Test MSE: 20.8416
KNN (k=15) Test MSE: 29.1910
```

In Scenario 1,
As the neighborhood size (k) increases from 1, the model smooths out the local noise, reducing variance. The Test MSE drops from 1.97 ($k = 1$) to a minimum of 1.27 ($k = 9$). However, when k increases further to 15, the model becomes too rigid i.e. starts to underfit, and the MSE begins to rise again to 1.34.
In Scenario 2,
The KNN model hits its optimal bias-variance balance at $k = 5$ with an MSE of 13.92 . At $k = 1$, it captures too much noise (MSE = 23.24). As k grows too large ($k = 15$), the model drastically over-smooths the true mathematical curves, valleys, and peaks of the data, leading to severe underfitting and a spiked MSE of 29.19.